

---

字符串string

# 复习

---

- |                |           |
|----------------|-----------|
| 1. 字符数组定义与声明   | 熟悉        |
| 2. 字符和字符串的区别   | 了解        |
| 3. 字符数组的地址     | 熟悉        |
| 4. 字符数组的输入输出   | <b>重点</b> |
| 5. 字符数组操作      | <b>重点</b> |
| 6. 字符串到数值类型的转换 | <b>重点</b> |
| 7. 字符检查        | <b>重点</b> |

# 目录

---

- |                      |    |
|----------------------|----|
| 1. C++ 中的string类     | 熟悉 |
| 2. string 类几种常见的构造函数 | 了解 |
| 3. string的常见操作       | 重点 |

# C++ 中的string类

---

C++ 标准库提供了 string 类类型，支持上述所有的操作，另外还增加了其他更多的功能。我们将学习 C++ 标准库中的这个类，现在让我们先来看看下面这个实例：

现在您可能还无法透彻地理解这个实例，因为到目前为止我们还没有讨论类和对象。所以现在您可以只是粗略地看下这个实例，等理解了面向对象的概念之后再回头来理解这个实例。

# 实例

```
#include <iostream>
#include <string>
using namespace std;
int main ()
{
    string str1 = "Hello";
    string str2 = "World";
    string str3;
    int len ;

    // 复制 str1 到 str3
    str3 = str1;
    cout << "str3 : " << str3 << endl;

    // 连接 str1 和 str2
    str3 = str1 + str2;
    cout << "str1 + str2 : " << str3 << endl;

    // 连接后，str3 的总长度
    len = str3.size();
    cout << "str3.size() : " << len << endl;

    return 0;
}
```



```
str3 : Hello
str1 + str2 : HelloWorld
str3.size() : 10
```

# string 类几种常见的构造函数

---

C++ 中提供了专门的头文件 `string`（注意不是 `string.h`，这个是 C 风格字符串相关函数的头文件），来支持 `string` 类型。`string` 类定义隐藏了字符串的数组性质，让我们可以像处理普通变量那样处理字符串。`string` 对象和字符数组之间的主要区别是：可以将 `string` 对象声明为简单变量，而不是数组。

# 构造函数

1 ) `string(const char *s)` : 将 `string` 对象初始化为 `s` 指向的字符串

```
string str("Hello!");
```

2 ) `string(size_type n,char c)` : 创建一个包含 `n` 个元素的 `string` 对象 , 其中每个元素都被初始化为字符 `c`

```
string str(10,'a');
```

3 ) `string(const string &str)` : 将一个 `string` 对象初始化为 `string` 对象 `str` ( 复制构造函数 )

```
string str1("hello!");
```

```
string str2(str1);
```

4 ) `string()` : 创建一个默认的 `string` 对象 , 长度为 0 ( 默认构造函数 )

```
string str;           // 创建一个空的 string 对象
```

`string` 类的设计允许程序自动处理 `string` 的大小 , 因此 , 上述代码创建了一个长度为 0 的 `string` 对象 , 但是向 `str` 中写入数据时 , 程序会自动调整 `str` 的长度。因此 , 与使用数组相比 , 使用 `string` 对象更方便 , 也更安全。

# string的常见操作

## 一、获取 string 对象的长度

在 C 语言中，使用 `strlen` 函数获取字符串的长度。在 C++ 中，可以使用 `string.size()` 函数或 `string.length()` 函数来获得 `string` 对象的长度。在 C++ 标准库中，两者的源代码如下：

```
size_type __CLR_OR_THIS_CALL length() const
{
    // return length of sequence
    return (_Mysize);
}
```

```
size_type __CLR_OR_THIS_CALL size() const
{
    // return length of sequence
    return (_Mysize);
}
```

# string长度

---

可见，这两个方法是完全一样的，并没有区别。length() 方法是 C 语言习惯保留的，size() 方法则是为了兼容 STL 容器而引入的。

```
string str("Hello,World!");
```

```
int strLen1 = str.length();
```

```
int strLen2 = str.size();
```

# string的常见操作

## 二、复制 string 对象

在 C 语言中，使用 `strcpy`、`strncpy` 函数来实现字符串的复制。在 C++ 中则方便很多，可以直接将一个 `string` 对象赋值给另一个 `string` 对象，即：

```
string str1("Hello,World!");  
string str2;  
str2 = str1;
```

由于 `string` 类会自动调整对象的大小，因此不需要担心目标数组不够大的问题。

# string的常见操作

## 三、string 对象的拼接和附加

在 C 语言中，使用 `strcat`、`strncat` 函数来进行字符串拼接操作。在 C++ 中也有多种方法来实现字符串拼接和附加操作：

### 1.使用 + 操作符拼接两个字符串

```
string str1("hello ");  
string str2("world!");  
string str3 = str1 + str2;
```

### 2.使用 += 操作符在字符串后面附加内容

可以使用 += 来在一个 string 对象后面附加一个 string 对象、字符以及 C 风格的字符串：

```
string str1("hello ");  
string str2("world!\n");  
str1 += str2;  
str1 += "nice job\n";  
str1 += 'a';
```

# string的常见操作

---

## 三、string 对象的拼接和附加

### 3.使用 `string.append()` 函数

可以使用 `string.append()` 函数来在一个 `string` 对象后面附加一个 `string` 对象或 C 风格的字符串：

```
string str1 = "hello,world!";
```

```
string str2 = "HELLO,WORLD!";
```

```
str1.append(str2);
```

```
str1.append("C string");
```

# string的常见操作

---

## 三、string 对象的拼接和附加

### 3.使用 `string.push_back()` 函数

可以使用 `string.push_back()` 函数来在一个 `string` 对象后面附加一个字符串：

```
string str("Hello");
```

```
str.push_back("abc");
```

# string的常见操作

## 四、string 对象的比较

在 C 语言中，使用 strcmp、strncmp 函数来进行字符串的比较。在 C++ 中，由于将 string 对象声明为了简单变量，故而对字符串的比较操作十分简单了，直接使用关系运算符（==、!=、<、<=、>、>=）即可：

```
string str1("hello");
```

```
string str2("hello");
```

```
if (str1 == str2)
```

```
    cout << "str1 = str2" << endl;
```

```
else if (str1 < str2)
```

```
    cout << "str1 < str2" << endl;
```

```
else
```

```
    cout << "str1 > str2" << endl;
```

# string的常见操作

## 四、string 对象的比较

使用类似 strcmp 的函数来进行 string 对象的比较，string 类提供的是 string.compare() 方法，函数原型如下：

```
int compare(const string&str) const;
```

```
int compare ( size_t pos , size_t len , const string&str ) const;
```

```
int compare ( size_t pos , size_t len , const string&str, size_t subpos , size_t sublen ) const;
```

```
int compare ( const char * s ) const;
```

```
int compare ( size_t pos , size_t len , const char * s ) const;
```

```
int compare ( size_t pos , size_t len , const char * s , size_t n ) const;
```

compare 方法的返回值如下：

- 1) 返回 0，表示相等；
- 2) 返回结果小于 0，表示比较字符串中第一个不匹配的字符比源字符串小，或者所有字符都匹配但是比较字符串比源字符串短；
- 3) 返回结果大于 0，表示比较字符串中第一个不匹配的字符比源字符串大，或者所有字符都匹配但是比较字符串比源字符串长。

# string的常见操作

## 四、string 对象的比较

```
string str1 ("aBcdef");
```

```
string str2 ("AbcdEf");
```

```
string str3 ("123456");
```

```
string str4 ("123dfg");
```

```
//下面是各种比较方法
```

```
int m=str1.compare (str2); //完整的A和B的比较
```

```
int n=str1.compare(1,5,str2); //"Bcdef"和"AbcdEf"比较
```

```
int p=str1.compare(1,5,str2,4,2); //"Bcdef"和"Ef"比较
```

```
int q=str3.compare(0,3,str4,0,3); //"123"和"123"比较
```

# string的常见操作

## 五、使用 string.substr() 函数来获取子串

可以使用 string.substr() 函数来获取子串，string.substr() 函数的定义如下：

```
string substr ( size_t pos = 0 , size_t len = npos ) const;
```

其中，pos 是子字符串的起始位置（索引，第一个字符的索引为 0），len 是子串的长度。这个函数的功能是：复制一个 string 对象中从 pos 处开始的 len 个字符到 string 对象 substr 中去，并返回 substr。

```
string str("Hello,World!");  
string subStr = str.substr(3,5);  
cout << subStr << endl;
```

这段代码的输出结果为："lo,Wo"。

# string的常见操作

## 六、访问 string 字符串的元素

可以像 C 语言中一样，将 string 对象当做一个数组，然后使用数组下标的方式来访问字符串中的元素；也可以使用 string.at(index) 的方式来访问元素（索引号从 0 开始）：

```
string str("Hello,World!");
```

```
cout << str[1] << endl;
```

// 使用数组下标的方式访问 string 字符串的元素

```
cout << str.at(1) << endl;
```

// 使用 at 索引访问 string 字符串的元素

# string的常见操作



npos -1  
不存在的位置

## 七、string 对象的查找操作

1.使用 `string.find()` 方法查找字符 find 方法的函数原型如下：

1) 从字符串的 pos 位置开始（若不指定 pos 的值，则默认从索引 0 处开始），查找子字符串 str。如果找到，则返回该子字符串首次出现时其首字符的索引；否则，返回 `string::npos`：

```
size_type find (const string& str, size_type pos = 0) const;
```

2) 从字符串的 pos 位置开始（若不指定 pos 的值，则默认从索引 0 处开始），查找子字符串 s。如果找到，则返回该子字符串首次出现时其首字符的索引；否则，返回 `string::npos`：

```
size_type find (const char *s, size_type pos = 0) const;
```

3) 从字符串的 pos 位置开始（若不指定 pos 的值，则默认从索引 0 处开始），查找 s 的前 n 个字符组成的子字符串。如果找到，则返回该子字符串首次出现时其首字符的索引；否则，返回 `string::npos`：

```
size_type find (const char *s, size_type pos, size_type n);
```

4) 从字符串的 pos 位置开始（若不指定 pos 的值，则默认从索引 0 处开始），查找字符 ch。如果找到，则返回该字符首次出现的位置；否则，返回 `string::npos`：

```
size_type find (char ch, size_type pos = 0) const;
```

# find 案例

```
#include <string>
#include <iostream>

using namespace std;

int main()
{
    string str("cat,dog,cat,pig,little cat,hotdog,little pig,angry dog");
    size_t catPos = str.find("cat",0);

    if (catPos == string::npos) {
        printf("没有找到字符串\n");
        return 0;
    }

    while (catPos != string::npos) {
        cout << "在索引 " << catPos << " 处找到字符串" << endl;
        catPos = str.find("cat", catPos + 1);
    }

    return 0;
}
```

# string的常见操作

## 七、string 对象的查找操作

### 2.string.rfind()

string.rfind() 与 string.find() 方法类似，只是查找顺序不一样，string.rfind() 是从指定位置 pos（默认为字符串末尾）开始向前查找，直到字符串的首部，并返回第一次查找到匹配项时匹配项首字符的索引。换句话说，就是查找子字符串或字符最后一次出现的位置。

# string的常见操作

## 七、string 对象的查找操作

### 3.string.find\_first\_of()

string.find\_first\_of() 方法在字符串中从指定位置开始向后（默认为索引 0 处）查找参数中任何一个字符首次出现的位置。

```
string str("catjdkjfellmlm");
int pos = str.find_first_of("gat");
if (pos == string::npos) {
    printf("没有匹配到\n");
    return 0;
}
else
    cout << "在索引 " << pos << " 处匹配到" << endl;
```

程序输出结果是：在索引 1 处匹配到。所查找的字符串 gat 中，第一次出现在字符串 str 中的字符是 'a'，该字符在 str 中的索引是 1。

# string的常见操作

## 七、string 对象的查找操作

### 4 `string.find_last_of()`

`string.find_last_of()` 方法在字符串中查找参数中任何一个字符最后一次出现的位置（也就是从指定位置开始往前查找，第一个出现的位置）。

### 5 `string.find_first_not_of()`

`string.find_first_not_of()` 方法在字符串中查找第一个不包含在参数中的字符。

### 6 `string.find_last_not_of()`

`string.find_last_not_of()` 方法在字符串中查找最后一个不包含在参数中的字符（从指定位置开始往前查找，第一个不包含在参数中的字符）。

# string的常见操作

## 八、string 对象的插入操作

使用 `string.insert()` 进行插入操作，函数原型如下：

```
string&insert ( size_t pos , const string&str );           // 在位置 pos 处插入字符串 str
string&insert ( size_t pos , const string&str , size_t subpos , size_t sublen ); // 在位置 pos 处插入字符串 str 的从位置 subpos 处开始的 sublen 个字符
string&insert ( size_t pos , const char * s );             // 在位置 pos 处插入字符串 s
string&insert ( size_t pos , const char * s , size_t n ); // 在位置 pos 处插入字符串 s 的前 n 个字符
string&insert ( size_t pos , size_t n , char c );           // 在位置 pos 处插入 n 个字符 c
iterator insert (const_iterator p, size_t n, char c);      // 在 p 处插入 n 个字符 c，并返回插入后迭代器的位置
iterator insert (const_iterator p, char c);                // 在 p 处插入字符 c，并返回插入后迭代器的位置
```

# insert案例

---

```
string str("abcdefgh");
```

```
str.insert(1,"test");    // 在位置 1 处插入字符串 "test"
```

```
cout << str << endl;
```

```
str.insert(10, 5, 'H');  // 在位置 10 处插入 5 个字符 'H'
```

```
cout << str << endl;
```

# string的常见操作

## 九、string 对象的删除操作

使用 string.erase() 进行元素删除操作

函数原型如下：

```
string& erase (size_t pos = 0, size_t len = npos);           // 删除从 pos 处开始的 n 个字符
```

```
iterator erase (const_iterator p);   // 删除 p 处的一个字符，并返回删除后迭代器的位置
```

```
iterator erase (const_iterator first, const_iterator last); // 删除从 first 到 last 之间的字符，并返回删除后迭代器的位置
```

# erase案例

---

```
string str("abc123efhuipj");
```

```
str.erase(5,6);           // 删除从索引位置 5 开始的 6 个字符
```

```
cout << "str 为 : " << str << endl;
```

# string的常见操作

---

## 十、判断字符串是否为空

使用 empty() 函数判断字符串是否为空

```
string str;  
if(str.empty()){  
    cout << "字符串为空" << endl;  
}
```

# string的常见操作

---

## 十一、使用 swap 函数交换两个字符串

```
string str1 = "abc";
```

```
string str2 = "hjkui";
```

```
str1.swap(str2);
```

```
cout << str1 << endl;
```

```
cout << str2 << endl;
```

# string的常见操作

---

## 十二、输入输出

```
string str1,str2;  
//读入方式1遇到换行停止  
getline(cin, str1);  
//读入方式2遇到空格停止  
cin>>str2;
```

```
//输出方式1  
printf("%s\n",str1.c_str());  
//输出方式2  
cout<<str2<<endl;
```

# string的常见操作

## 十三、字符串替换

```
string &replace(int p0, int n0,const char *s);  
//删除从p0开始的n0个字符，然后在p0处插入串s
```

```
string &replace(int p0, int n0,const char *s, int n);  
//删除p0开始的n0个字符，然后在p0处插入字符串s的前n个字符
```

```
string &replace(int p0, int n0,const string &s);  
//删除从p0开始的n0个字符，然后在p0处插入串s
```

```
string &replace(int p0, int n0,const string &s, int pos, int n);  
//删除p0开始的n0个字符，然后在p0处插入串s中从pos开始的n个字符
```

```
string &replace(int p0, int n0,int n, char c);  
//删除p0开始的n0个字符，然后在p0处插入n个字符c
```

# replace案例

---

```
string s1 = "abcdefghijklmn";  
s1.replace(1, 3, "HK");  
cout << s1 << endl; //aHKdefghijklmn
```

```
string s2 = "abcdefghijklmn";  
s2.replace(2, 1, "qwerty", 5);  
cout << s2 << endl; //abqwertydefghijklmn
```

```
string s3 = "hjkbnm";  
s3.replace(1,2,3,'a');  
cout << s3 << endl; //haaabnm
```

# string的常见操作

## 十四、字符串反转

使用algorithm 头文件里面的reverse函数

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {

    string s1;
    cin >> s1;

    reverse(s1.begin(), s1.end());
    cout << s1 << endl;

    return 0;
}
```

# 课堂练习1 题目

---

从键盘输入一个字符串（由英文字母组成），将该字符串中所有的字母改成该字母的下一个字母，如b改成c,字母z改为a要求，大小写不变。

输入用例：abcdz

输出用例：bcdea

# 课堂练习1 代码

---

```
string s1,s2;
cin >> s1;

int n = s1.size();
for (int i = 0; i < n; i++)
{
    if (s1.at(i) == 'z') {
        s2 += 'a';
    }
    else {

        s2 += s1.at(i) + 1;
    }

}

cout << s2 << endl;
```

# 课堂练习2 题目

---

给定一个字符串  $s$ ，找到它的第一个不重复的字符，并返回它的索引。如果不存在，则返回 -1

输入用例：abccga

输出用例：1

# 课堂练习2 代码

```
string s1;
cin >> s1;

int n = s1.size();
int cnt[26] = { 0 };

for (int i = 0; i < n; i++) {
    cnt[s1[i] - 'a']++;
}

for (int i = 0; i < n; i++) {
    if (cnt[s1[i] - 'a'] == 1) {
        cout << i << endl;
        return 0;
    }
}
cout << -1 << endl;
```

# 知识点总结

---

1. C++ 中的string类

熟悉

2. string 类几种常见的构造函数

了解

3. string的常见操作

重点

# 作业

---

## 1. 从字符串中提取出来数字

输入用例：abc123edf

输出用例：123

# 下节课预告

---

## 综合案例2